

Megalodon → Lean 4 Translator: Code Walkthrough

`sexprinfo-prototype/sexpr_translate.py` and `pf_to_sexpr.patch`, in the megalodon-lean4 repository. Steps 0–7.

Step	Lines	What it covers
0		The big picture: how the pieces fit
1	patch	The one-line change to Megalodon
2		What the input looks like (a real example)
3	19–63	Parsing
4	103–215	Translating types, terms and proofs
5	291–393	The driver
6	219–278	The safety audit
7	66–100, 396–465	Output files and categories

Step 0: The big picture

#	Stage	What happens
1	100thms_12.mg	Megalodon source file: 999 theorems with proofs
2	patched Megalodon	checks every proof, exports each checked proof term
3	corpus.sexpr	the export: one S-expression per declaration, one per line
4	sexpr_translate.py	this file: maps each S-expression to Lean 4 syntax
5	All.lean	the Lean 4 output: every definition, theorem and proof
6	Lean kernel	re-checks every proof independently

The key design decision: **we do not translate Megalodon's source text**. We translate the **proof terms Megalodon has already checked**. Megalodon parses the `.mg` file, checks every proof, and exports the checked result in a simple structured format (S-expressions). The Python script only has to map that format onto Lean, one construct at a time.

Why that matters: the older translator (`megalodon_full.py`, 529/999) parsed the source text with regular expressions. Source text is ambiguous (parentheses, commas, operator precedence), and every bug it had came from that. The structured export has no surface syntax left to misread, so that whole class of bug cannot occur here.

Trust. Lean re-checks every output proof with its own kernel. The translator does not need to be trusted to produce *valid* proofs: a wrong proof term would be rejected by

Lean. The one remaining gap is a translator that consistently mistranslates *statements*, which Lean cannot detect; Step 6 describes the audit that addresses it.

Result

- **999 / 999 theorems** translated and accepted by Lean's kernel, with zero sorry and zero sorryAx (no step skipped or assumed).
- Checked two ways: a full recompile of the output, and a separate `#print axioms` pass over every theorem, confirming each proof relies only on Megalodon's own 17 foundational axioms (e.g. `set_ext`, `prop_ext`, `Eps_i_ax`).
- Includes every surreal-number theorem: 606 under this translator's categorisation (Step 7), a category where the older text-based translator reached only 23–24%.
- Reproducible: re-running the translator on the same export regenerates the verified `All.lean` byte for byte.
- The AI-mathematician project runs on this output: its `lib/MegLib.lean` is a copy of the verified `All.lean`.

Step 1: The patch to Megalodon (`pf_to_sexpr.patch`)

Megalodon already had an `-sexprinfo` mode that exported statements, and a function `pf_to_sexpr` that can print a proof term; it just never printed proofs in that mode. The patch changes **one line** of `src/megalodon.ml` (line 2364):

```
| Some(dl) ->
  deltaset := dl;
  if !sexprinfo then (
    List.iter (fun d -> Printf.printf "(DELTA \"%s\")\n" d) dl;
+   Printf.printf "(PROOF \"%s\" %s)\n" thmname (pf_to_sexpr dgpf);
    Printf.printf "(QED)\n");
```

- `Some(dl)` is the branch Megalodon reaches **only after the proof has checked**. The line just above it raises "Proof doesn't prove the proposition." if it fails.
- `dgpf` is the checked proof term, and `pf_to_sexpr` is Megalodon's own existing printer.
- Each theorem now gets a line `(PROOF "name" <term>)`, so **only proofs Megalodon accepted are ever exported**.

Step 2: What the input looks like

Three real lines from the export (hashes shortened), using the theorem `andI` ("from A and B, conclude $A \wedge B$ ") as the running example:

```
(DEF "and" "87fba1..." 0 (AR (PROP) (AR (PROP) (PROP))))
```

```

(LAM (PROP) (LAM (PROP)
  (ALL (PROP) (IMP (IMP (DB 2) (IMP (DB 1) (DB 0))) (DB 0))))))
(THM "andI" "7f6246..." "3bf198..." 0
  (ALL (PROP) (ALL (PROP)
    (IMP (DB 1) (IMP (DB 0)
      (AP (AP (TMH "87fba1...") (DB 1)) (DB 0)))))))
(PROOF "andI"
  (TLAM (PROP) (TLAM (PROP) (PLAM (DB 1) (PLAM (DB 0)
    (TLAM (PROP) (PLAM (IMP (DB 2) (IMP (DB 1) (DB 0)))
      (PPFAP (PPFAP (HYP 0) (HYP 2)) (HYP 1))))))))))

```

Three ideas are needed to read these.

1. De Bruijn indices (DB n). Variables have no names, only a number: "the binder n steps out from here". DB 0 is the nearest enclosing ALL/LAM, DB 1 the next one out. In andI's statement, ALL (PROP) ALL (PROP) IMP (DB 1) (IMP (DB 0) ...) means "for all A, for all B: $A \rightarrow B \rightarrow \dots$ ", where DB 1 is A and DB 0 is B. There is therefore no name clash or scoping ambiguity; the translator invents fresh names (x69, x70) when it writes Lean.

2. References by hash (TMH). Earlier declarations are referenced by a **content hash**, not by name. 87fba1... is the hash on the DEF "and" line, so (AP (AP (TMH "87fba1...") (DB 1)) (DB 0)) is and A B. The translator keeps a table from hash to name.

3. Proofs have their own constructors:

- TLAM: "for any A, ...", which becomes `fun x : Prop =>.`
- PLAM: "assume a hypothesis h : ...", which becomes `fun h : ... =>.`
- HYP n : a De Bruijn index for *hypotheses*, with its own separate count.
- PPFAP: apply one proof to another.

Megalodon defines and the classical higher-order way: $\mathbf{A \wedge B := \forall P, (A \rightarrow B \rightarrow P) \rightarrow P}$. So the proof of andI reads: take A, B, a proof a of A and a proof b of B; then for any P and any $f : A \rightarrow B \rightarrow P$, return $f\ a\ b$. That is exactly the final part, (PPFAP (PPFAP (HYP 0) (HYP 2)) (HYP 1)): f applied to a , then to b (HYP 0 is f , the most recent hypothesis; HYP 2 is a ; HYP 1 is b).

The translator's output, taken from the verified All.lean:

```

theorem andI :
  (forall x69 : Prop, (forall x70 : Prop,
    (x69 -> (x70 -> ((and x69) x70)))) :=
  (fun x71 : Prop => (fun x72 : Prop =>
    (fun h73 : x71 => (fun h74 : x72 =>
      (fun x75 : Prop => (fun h76 : (x71 -> (x72 -> x75)) =>
        ((h76 h73) h74)))))))

```

Line breaks added for readability; the file has it on one line.

Every piece lines up one-to-one with the S-expression: TLAM gives `fun x71 : Prop`, PLAM gives `fun h73 : x71`, and the end is `h76 h73 h74`, i.e. $f\ a\ b$.

Step 3: Parsing (lines 19–63)

The job is to turn text such as (THM "andI" "7f62..." 0 (ALL (PROP) ...)) into nested Python lists: ["THM", "andI", "7f62...", "0", ["ALL", ["PROP"], ...]].

parse_sexpr(text, i) — lines 19–47

Reads one expression starting at position *i* and returns (node, next position). It handles three cases.

```
while text[i].isspace():      # 22-23: skip spaces
    i += 1
if text[i] == '(':            # 24-33: a LIST
    i += 1
    items = []
    while True:
        ...skip spaces...
        if text[i] == ')':
            return items, i + 1      # end of list
        node, i = parse_sexpr(text, i) # recursion: parse one child
    items.append(node)
```

A list, (...) (line 24): keep parsing children, calling itself recursively, until the matching). Recursion is what handles nesting of any depth; some surreal-number proofs are nested thousands of levels deep.

```
elif text[i] == '"':          # 34-42: a QUOTED STRING, e.g. "andI"
    ...
    if text[j] == '\\':        # backslash escape: take the next char literally
        buf.append(text[j+1]); j += 2
```

A quoted string (line 34): collect characters up to the closing quote. A backslash means "take the next character literally", in case a name contains a quote.

```
else:                          # 43-47: a bare ATOM, e.g. THM, ALL, 0
    while j < len(text) and not text[j].isspace() and text[j] not in "()":
        j += 1
    return text[i:j], j
```

An atom (line 43): read until a space or a bracket. Numbers such as 0 remain strings here and are converted later with int().

parse_toplevel_forms(text) — lines 50–63

```
for line in text.splitlines():
    line = line.strip()
    if not line.startswith("("): # skip non-data lines (e.g. WARNING: ...)
        continue
    try:
        node, _ = parse_sexpr(line, 0)
    except Exception:
        continue
    forms.append(node)
```

Megalodon writes **one declaration per line**, so each line is parsed independently. Lines that are not S-expressions, such as Megalodon's own warnings, are skipped.

Known rough edge. The except `Exception: continue` on line 60 silently skips any line that fails to parse. For this corpus it is provably harmless: a theorem dropped here would be missing from `All.lean`, and the 999/999 count confirms none were lost. On a future corpus, though, it could drop declarations without warning; counting and reporting skipped lines would close that gap.

Step 4: The heart of the translator (lines 103–215)

This is the part that does the actual translation, and it is only about 110 lines: a context object and three functions, one for each layer of Megalodon's language.

Layer	Examples in Megalodon	Function
Types	<code>set, Prop, set → Prop</code>	<code>tp_to_lean</code>
Terms (the statements)	<code>and A B, ∀x, ..., fun x => ...</code>	<code>tm_to_lean</code>
Proofs	<code>"assume h ...", "apply h to a"</code>	<code>pf_to_lean</code>

4a. The context, Ctx (lines 103–127)

```
class Ctx:
  def __init__(self):
    self.hash_to_name = {} # TMH/KNOWN hash -> Lean name
    self.prim_to_name = {} # PRIM index -> Lean name
    self.tp_stack = [] # bound type-var names (TPVAR De Bruijn)
    self.tm_stack = [] # bound term-var names (DB De Bruijn)
    self.pf_stack = [] # bound proof-var names (HYP De Bruijn)
    self.counter = 0
    self.current_deps = None
```

- **Two lookup tables** answer "which declaration does this reference mean?".
`hash_to_name` covers references by content hash (TMH/KNOWN, Step 2).
`prim_to_name` covers Megalodon's 6 built-in set-theory primitives `In`, `Empty`, `Union`, `Power`, `Repl` and `UnivOf`, which Megalodon references by number (PRIM 1 is `In`, and so on).
- **Three stacks** answer "which bound variable does DB n mean?". There are three because Megalodon numbers three kinds of variable separately: type variables (TPVAR n), term variables (DB n) and proof hypotheses (HYP n).
- **fresh(base)** (lines 113–115) makes a new name such as `x71` or `h73` from a single counter shared by the whole file, so every generated name is unique and names can never clash.

- **resolve_hash, resolve_prim** (lines 117–127) look up a name and also record it in `current_deps`, the set of declarations this entry uses (used in Step 7).

The De Bruijn trick, which the whole file relies on: `stack[-(n + 1)]`. Entering a binder pushes a fresh name onto the stack; leaving it pops the name. The top of the stack is therefore always the innermost binder: `DB 0` is `stack[-1]`, `DB 1` is `stack[-2]`, and `DB n` is `stack[-(n+1)]`. One line resolves every variable, with no scoping logic at all.

4b. Types, `tp_to_lean` (lines 130–140)

```
if tag == "TPVAR": return ctx.tp_stack[-(int(node[1]) + 1)] # type parameter
if tag == "PROP": return "Prop"
if tag == "SET":  return "set"
if tag == "AR":  return f"({tp(node[1])} -> {tp(node[2])})" # function type
```

Recursive calls shortened to `tp(...)` for space.

That is the whole type language: `Prop`, `set`, function types (`AR`, "arrow"), and type parameters (`TPVAR`) for polymorphic definitions such as equality. `set` is declared once in the output header as `axiom set : Type`. Any other tag raises `NotImplementedError` (line 140), so unexpected input fails loudly instead of producing wrong output.

4c. Terms, `tm_to_lean` (lines 143–174)

Tag	Meaning	Lean output	Lines
DB <i>n</i>	bound variable	<code>tm_stack[-(n+1)]</code>	145–146
TMH <i>h</i>	earlier declaration, by hash	its name	147–151
PRIM <i>i</i>	built-in primitive	its name	152–153
AP <i>f a</i>	application	<code>(f a)</code>	154–155
LAM <i>T b</i>	function	<code>(fun x : T => b)</code>	156–162
IMP <i>a b</i>	implication	<code>(a -> b)</code>	163–164
ALL <i>T b</i>	for all	<code>(forall x : T, b)</code>	165–171
TPAP <i>f T</i>	polymorphic constant at a type	<code>(f T)</code> , e.g. <code>(eq set)</code>	172–173

The binder cases (`LAM`, `ALL`) all follow the same four-step pattern:

```
v = ctx.fresh("x")           # 1. invent a fresh name
ctx.tm_stack.append(v)       # 2. push it: the body's DB 0 now means v
body = tm_to_lean(node[2], ctx) # 3. translate the body (recursion)
ctx.tm_stack.pop()           # 4. pop it: back to the outer scope
```

- **Everything is fully parenthesised.** Every application, arrow and binder gets its own brackets, so Lean never has to decide operator precedence. It is ugly to read but cannot be misread.

- **Lines 149–150 check for forward references.** If a hash is not in the table yet, translation stops with an error instead of guessing. Because declarations are processed in file order, this also enforces that every declaration only uses things declared before it.

4d. Proofs, `pf_to_lean` (lines 177–215)

This rests on the **Curry-Howard correspondence**, the principle Lean itself is built on: a proof is a program. A proof of $A \rightarrow B$ is a function from proofs of A to proofs of B , and a proof of $\forall x, P x$ is a function from x to proofs of $P x$. Proofs therefore translate into the same fun and application syntax as terms.

Tag	Proof step	Lean output	Lines
HYP n	use hypothesis n	<code>pf_stack[-(n+1)]</code>	179–180
KNOWN h	use an earlier theorem, by hash	its name	181–185
PLAM $A \ p$	assume $h : A$, then prove ...	<code>(fun h : A => p)</code>	190–196
PPFAP $p \ q$	modus ponens: apply p to q	<code>(p q)</code>	188–189
TLAM $T \ p$	$\forall$ -introduction: take any $x : T$	<code>(fun x : T => p)</code>	197–206
PTMAP $p \ t$	$\forall$ -elimination: instantiate at t	<code>(p t)</code>	186–187
PTPLAM p	for any type T (polymorphic)	<code>(fun T : Type => p)</code>	207–212
PTPAP $p \ T$	instantiate a polymorphic proof	<code>(p T)</code>	213–214

The subtle one: TLAM (comment on lines 198–200). TLAM introduces a term variable inside a proof, so it pushes onto `tm_stack`, the term stack, not a separate proof-level stack. It has to: the hypotheses that follow refer to that variable with ordinary DB indices. In `andI`, after two TLAMs comes PLAM (DB 1), "assume a proof of A ", where A is the first TLAM's variable, reached through DB 1. With its own stack, DB 1 would point at the wrong variable. The name makes it easy to confuse with PTPLAM, which really is about types, hence the comment.

4e. Worked trace: `andI` through `pf_to_lean`

The statement was translated first and used `x69/x70`, so the counter continues from 71.

Node	Action	tm_stack	pf_stack
TLAM (PROP)	fresh x71 , push	x71	
TLAM (PROP)	fresh x72 , push	x71 x72	
PLAM (DB 1)	type = DB 1 = x71 ; fresh h73	x71 x72	h73
PLAM (DB 0)	type = x72 ; fresh h74	x71 x72	h73 h74
TLAM (PROP)	fresh x75 , push	x71 x72 x75	h73 h74

Node	Action	tm_stack	pf_stack
PLAM (IMP (DB 2) (IMP (DB 1) (DB 0)))	type = (x71 -> (x72 -> x75)); fresh h76	x71 x72 x75	h73 h74 h76
PPFAP (PPFAP (HYP 0) (HYP 2)) (HYP 1)	HYP 0 = h76 , HYP 2 = h73 , HYP 1 = h74		

The result is ((h76 h73) h74) inside six funs: exactly the verified output shown in Step 2, name for name.

Step 5: The driver, translate(forms) (lines 291–393)

The driver goes through the parsed file **in order**, one declaration at a time, and sends each part to the Step 4 functions.

Megalodon tag	Count	Becomes in Lean	Lines
PARAM (typed constant)	1	axiom name : type	321–330
PRIM (built-in, e.g. In)	6	axiom name : type + registers its number	352–359
AXIOM	13	axiom name : statement	331–340
DEF	139	noncomputable def name : type := body	341–351
THM (statement)	999	stored, not written yet	360–367
PROOF	999	theorem name : statement := proof	368–381

Every branch follows the same pattern:

1. Reset current_deps.
2. Push any type parameters.
3. Translate with Step 4.
4. Pop the parameters.
5. Append the Lean text.
6. **Only then** register the name in hash_to_name.

That last point is deliberate. A declaration's own hash is not registered until it has been translated, so a proof that cites its own theorem fails with the "unresolved hash" error from Step 4. Together with the forward-reference check, the output cannot contain circular reasoning.

Why THM and PROOF are handled separately (lines 360–381). Megalodon emits the statement (THM), then the proof (PROOF) a few lines later. The driver translates the statement first and stores it in thm_stmt. When the proof arrives, it continues the

dependency set the statement started (line 373), translates the proof, writes theorem name : statement := proof, assigns a category (Step 7) and registers the hash. A THM whose PROOF never arrives is simply never written (line 370); any later reference to it would fail loudly, and the 999/0 count confirms this never happened.

noncomputable def (line 349). Many definitions go through Megalodon's choice operator Eps_i, an axiom with no computational content. Lean will not compile code for those, so every definition is marked noncomputable. This affects code generation only, not logic.

Type parameters (lines 298–316). Four declarations are polymorphic: func_ext with 2 type parameters, and eq, neq, ex with 1 each. Lean writes them as explicit parameters:

```
axiom func_ext (T21 : Type) (T22 : Type) : (forall x23 : (T21 -> T22), ...
```

- **Explicit (T : Type), not implicit {T : Type} (lines 311–316).** Megalodon always applies a polymorphic constant to its type explicitly (the TPAP node), so the Lean side must take it explicitly too; hence (eq set) x y in the output.
- **The order (push_tpvars, lines 298–305).** The fresh names are pushed in reverse, so that TPVAR 0 resolves to the first Lean parameter: in func_ext, TPVAR 0 is T21 (domain) and TPVAR 1 is T22 (codomain). The full compile confirms it: func_ext is used 14 times, including 2 at (func_ext set) Prop, which would be type errors if the order were swapped.

Error handling (lines 382–392). Each declaration is wrapped in try/except. A failure is counted, up to 15 examples are printed, and translation carries on, so one problem does not hide others. On the real corpus the result is translated OK: 999 failed: 0.

Known rough edges

- **Latent: type-parameter order for proofs.** When a PROOF arrives, line 374 re-pushes the statement's type-parameter names without reversing them, while push_tpvars reversed them for the statement. For 0 or 1 parameters the two orders coincide; with 2 or more they would disagree. No theorem in this corpus has any type parameters (all 999 have 0), so this is never exercised and does not affect the 999/999 result. It is a one-line fix before using the translator on a corpus with polymorphic theorems.
- main() reports failures but still exits normally; a script that runs the translator automatically should check the "failed:" count.

Step 6: The safety audit (lines 219–278)

The problem it addresses. Lean checks that each translated proof proves its translated statement. It cannot check that the translated statement means the same as Megalodon's original. A translator that got a statement wrong, say by swapping two arguments, would produce a wrong statement plus a matching wrong proof, and Lean would accept both.

The idea. Megalodon gives every declaration a content hash computed from its statement, so two declarations with the same statement get the same hash whatever their names. That hash comes from Megalodon, independent of our translator, so it serves as ground truth.

1. **Lines 248–262:** collect every declaration's hash, with its name and raw statement (for theorems, the ahv hash from the THM line).
2. **Line 264:** find hashes claimed by more than one declaration. This corpus has 8 such pairs, e.g. `pair_Sigma` and `lamI`: different names, same statement.
3. **Lines 266–269:** check each pair's raw statements are structurally identical.
4. **Lines 437–442 (in main):** if any pair disagrees, refuse to translate and exit with an error.

Actual output: 1150 declarations, 8 share a hash with at least one other, 0 of those disagree on raw structure (1,158 declarations minus 8 duplicates gives 1,150 distinct hashes).

Why checking the raw input is enough. Translation is a deterministic function of the S-expression structure (Step 4), so identical input structure always gives identically shaped Lean output. The outputs would differ only in fresh names such as `x69` versus `x102`, which is why the code compares inputs rather than rendered Lean text.

Its limits (stated in the code's own docstring, lines 242–246). The audit catches inconsistent handling of statements Megalodon says are identical. It does not catch a mistake the translator makes the same way everywhere. What guards against that is the design: each construct maps to exactly one Lean construct (the Step 4 tables), a mapping small enough, about 110 lines, to check by reading. The `andI` trace in Step 4 is an example of such a check.

Step 7: Output files and categories (lines 66–100, 396–465)

HEADER and render (lines 396–401). The output file is:

```
set_option maxRecDepth 8000
namespace Megalodon
axiom set : Type
... every declaration, in source order ...
end Megalodon
```

- namespace `Megalodon` keeps names such as `and`, `or` and `True` from clashing with Lean's built-in versions.
- `maxRecDepth 8000` raises Lean's recursion limit, because two very deeply nested proofs needed it. It is set globally; a per-theorem setting would be more precise.

categorise (lines 66–100) sorts each theorem into one of five areas. It checks the theorem's name, translated statement and cited names against keyword patterns (`DOMAIN`, lines 69–85) **in order**: `surreals`, `ordinals`, `naturals`, `sets`. If none match, it checks logic vocabulary (`PROP_VOCAB`). As a last fallback, anything mentioning a number goes to `nat_arith`, everything else to `prop_logic`. Order matters: a surreal-number theorem usually mentions `sets` too, but belongs in `surreals`, so that is checked first. It is a vocabulary heuristic, copied from the older translator so the categories are comparable. The categories affect only how the output is split into files, never what is proved.

<code>prop_logic</code>	<code>set_theory</code>	<code>nat_arith</code>	<code>ordinals</code>	<code>surreals</code>	<code>total</code>
39	179	119	56	606	999

render_category (lines 404–428) writes one file per area that compiles on its own:

1. Start from that area's own theorems (line 409).
2. Follow their dependencies (`current_deps` from Step 4), then their dependencies, and so on (lines 410–419): a worklist loop computing the transitive closure of everything the area needs.
3. Keep everything needed, in original source order, so each item still comes after what it uses (line 421).
4. Mark borrowed theorems with a comment such as `-- dependency from set_theory` (lines 424–426).

main (lines 431–461) ties it together: read, parse (Step 3), audit and refuse on a mismatch (Step 6), translate (Step 5), write `All.lean` (authoritative) plus the 5 area files, and print the counts.

Verification of the output

- `lean All.lean` compiles with no errors.
- A `#print axioms` pass over every theorem confirms only Megalodon's 17 foundational axioms are used.
- Re-running the translator reproduces the verified `All.lean` byte for byte.